Chapter 21: Parent-Child Database Updates

In our last chapter we got the connection to our database working but our example was pretty simple. In this chapter we are going to step up our game by building three grids with parent-child data relationships. Let's jump to the inside and take a look at some code.

We begin with this image of the DDL I used to create this simple order processing schema. As shown in the figure, we have customers placing one or more orders, each of which has one or more lineitems. Each lineitem references a valid product and each order has a valid order status. (Figure 21.1)

Figure 21.1: Schema DDL

```
use model;
go

create table Customers
(
        Customer_ID                     int,
        Name                            varchar(50) not null,
        Email                           varchar(50) not null,
        Phone                           varchar(15) not null,
        Credit_Status                   int default -1,
        Delete_Status                   bit not null default 0,  -- 0 infers not deleted, 1 = deleted
        primary key (Customer_ID)
);
go

create table Order_Statuses
(
        Order_Status                    varchar(25) primary key,
        Delete_Status                   bit not null default 0,  -- 0 infers not deleted, 1 = deleted
);
go

create table Orders
(
        Order_ID                        int primary key,
        Order_Status                    varchar(25) not null,
        Order_Date                      DateTime default GETDATE(),
        Due_Date                        DateTime,
        Customer_ID                     int not null,
        Delete_Status                   bit not null default 0,  -- 0 infers not deleted, 1 = deleted
        constraint Orders_fk0 foreign key (Order_Status) references Order_Statuses(Order_Status),
        constraint Orders_fk1 foreign key (Customer_ID) references Customers(Customer_ID)
);
go

create table Products
(
        Product_ID                      int primary key,
        SKU                             varchar(15),
        Name                            varchar(50) not null,
        Price                           decimal(8,2),
        Delete_Status                   bit not null default 0,  -- 0 infers not deleted, 1 = deleted
);
go

create table Line_Items
(
        Order_ID                        int,
        Line_Item_Number                int,
        Product_ID                      int not null,
        Quantity                        int not null,
        Extended_Price                  decimal(8,2) not null,
        Delete_Status                   bit not null default 0,  -- 0 infers not deleted, 1 = deleted
        constraint Line_Items_pk primary key (Order_ID, Line_Item_Number),
        constraint Line_Items_fk0 foreign key (Product_ID) references Products(Product_ID)
);
go
```

I next of course wrote some DML to get some data into these tables. Again, super straight forward stuff here. (Figure 21.2)

Figure 21.2: DML

```
insert into Products(Product_ID, SKU, Name, Price, Delete_Status)
values (1, 'B4598', 'Big Bob''s Hot Sauce', 45.25, 0);
insert into Products(Product_ID, SKU, Name, Price, Delete_Status)
values (2, 'F9458', 'Flowers for Mary', 50.00, 0);
insert into Products(Product_ID, SKU, Name, Price, Delete_Status)
values (3, 'B98', 'A New Book by Rick', 69, 0);
insert into Products(Product_ID, SKU, Name, Price, Delete_Status)
values (4, 'B4598_v2', 'Big Bob''s Hot Sauce V2', 45.25, 0);
insert into Products(Product_ID, SKU, Name, Price, Delete_Status)
values (5, 'G6532', 'Gus''s Breakfast Treats', 4.25, 0);
go

insert into Order_Statuses (Order_Status, Delete_Status)
values ('New',0);
insert into Order_Statuses (Order_Status, Delete_Status)
values ('WIP',0);
insert into Order_Statuses (Order_Status, Delete_Status)
values ('Ready to Ship',0);
insert into Order_Statuses (Order_Status, Delete_Status)
values ('Closed',0);
insert into Order_Statuses (Order_Status, Delete_Status)
values ('Returned for Refund',0);
go

insert into Customers (Customer_ID, Name, Email, Phone, Credit_Status, Delete_Status)
values (1,'The Bar Next Door','someemail123@gmail.com','1 9198101234',1000,0);
insert into Customers (Customer_ID, Name, Email, Phone, Credit_Status, Delete_Status)
values (2,'Car Dealership','someemai1313@gmail.com','1 3218101234',1000,0);
insert into Customers (Customer_ID, Name, Email, Phone, Credit_Status, Delete_Status)
values (3,'Baseball Hall of Fame','email123@gmail.com','1 4568101234',1000,0);
insert into Customers (Customer_ID, Name, Email, Phone, Credit_Status, Delete_Status)
values (4,'Second Base','someemail789@gmail.com','1 4568101234',1000,0);
insert into Customers (Customer_ID, Name, Email, Phone, Credit_Status, Delete_Status)
values (5,'Hat Off To You','somee123@gmail.com','1 9878101234',1000,0);
go

insert into Orders (Order_ID, Order_Status, Order_Date, Due_Date, Customer_ID, Delete_Status)
values (1,'New', '20250428','20250528',1,0);
insert into Orders (Order_ID, Order_Status, Order_Date, Due_Date, Customer_ID, Delete_Status)
values (2,'Ready to Ship', '20250401','20250501',2,0);
insert into Orders (Order_ID, Order_Status, Order_Date, Due_Date, Customer_ID, Delete_Status)
values (3,'Returned for Refund', '20250528','20250628',3,0);
insert into Orders (Order_ID, Order_Status, Order_Date, Due_Date, Customer_ID, Delete_Status)
values (4,'WIP', '20250428','20250528',1,0);
go

insert into Line_Items (Order_ID, Line_Item_Number, Product_ID, Quantity, Extended_Price, Delete_Status)
values (1,1,1,10,1500.50,0);
insert into Line_Items (Order_ID, Line_Item_Number, Product_ID, Quantity, Extended_Price, Delete_Status)
values (1,2,3,5,1500.50,0);
insert into Line_Items (Order_ID, Line_Item_Number, Product_ID, Quantity, Extended_Price, Delete_Status)
values (1,3,2,10,1500.50,0);
insert into Line_Items (Order_ID, Line_Item_Number, Product_ID, Quantity, Extended_Price, Delete_Status)
values (2,1,1,10,1500.50,0);
go
```

The GUI for this example will include three DataGridView objects and a Submit button. The first DataGridView will support the customer information, the second will house the orders information and the final one will display the lineitem information. (Figure 21.3)

Figure 21.3: GUI Setup

```
1 reference
private void Form1_Load(object sender, EventArgs e)
{
    this.Size = new System.Drawing.Size(1200, 800);
    this.Text = "Customer Service";

    DataGridView CustomersGrid = new DataGridView();
    CustomersGrid.Name = "customersGridView";
    CustomersGrid.BorderStyle = BorderStyle.FixedSingle;
    CustomersGrid.Location = new System.Drawing.Point(25, 25);
    CustomersGrid.Size = new System.Drawing.Size(550, 250);
    this.Controls.Add(CustomersGrid);

    DataGridView OrdersGrid = new DataGridView();
    OrdersGrid.Name = "ordersGridView";
    OrdersGrid.BorderStyle = BorderStyle.FixedSingle;
    OrdersGrid.Location = new System.Drawing.Point(650, 25);
    OrdersGrid.Size = new System.Drawing.Size(500, 250);
    this.Controls.Add(OrdersGrid);

    DataGridView lineitemsGrid = new DataGridView();
    lineitemsGrid.Name = "lineitemsGridView";
    lineitemsGrid.BorderStyle = BorderStyle.FixedSingle;
    lineitemsGrid.Location = new System.Drawing.Point(25, 300);
    lineitemsGrid.Size = new System.Drawing.Size(1125, 350);
    this.Controls.Add(lineitemsGrid);

    Button submitButton = new Button();
    submitButton.Location = new System.Drawing.Point(550, 700);
    submitButton.BackColor = System.Drawing.Color.Red;
    submitButton.ForeColor = System.Drawing.Color.White;
    submitButton.Text = "Submit";
    submitButton.Size = new System.Drawing.Size(100, 30);
    this.Controls.Add(submitButton);

}
```

In this next figure we see my code for setting up the three DataTables. The only thing interesting here is how I have denormalized the lineitems query to include the product name. (Figure 21.4)

Figure 21.4: DataTable Setup



```
namespace chapter_21_parent_child_grids
{
    3 references
    public partial class Form1 : Form
    {
        DataTable tblCustomers, tblOrders, tblLineItems;
        DataSet tblDataSet;
        1 reference
        public Form1()
        {
            InitializeComponent();
        }

        1 reference
        private void Form1_Load(object sender, EventArgs e)
        {
            SqlConnection conn = new SqlConnection(@"Data Source = localhost\SQLEXPRESS; Initial Catalog = model;
                                Integrated Security = True; TrustServerCertificate = True;");

            SqlDataAdapter myDA1 = new SqlDataAdapter("select customer_id, name, email, phone, credit_status from customers order by customer_id", conn);
            tblCustomers = new DataTable("tblCustomers");
            myDA1.Fill(tblCustomers);
            SqlDataAdapter myDA2 = new SqlDataAdapter("select order_id, order_status, order_date from orders order by order_id", conn);
            tblOrders = new DataTable("tblOrders");
            myDA2.Fill(tblOrders);
            SqlDataAdapter myDA3 = new SqlDataAdapter("select a.order_id, a.line_item_number, a.product_id, b.name, a.quantity, a.extended_price " +
                                                      "from line_items a, products b " +
                                                      "where a.product_id = b.product_id " +
                                                      "order by a.order_id, a.line_item_number", conn);
            tblLineItems = new DataTable("tblLineItems");
            myDA3.Fill(tblLineItems);

            this.Size = new System.Drawing.Size(1200, 800);
            this.Text = "Customer Service";
```

In this next piece of code we introduce the DataSet class. The DataSet class is cool because it basically allows you to emulate a relational database in memory. The class supports two primary collections – tables and relations. The first parameter of the Relations property is noteworthy because it is actually just a string literal name for the relationship between the second and third parameters. (Figure 21.5)

Figure 21.5: DataSet



```
SqlDataAdapter myDA2 = new SqlDataAdapter("select order_id, order_status, order_date from orders order by order_id", conn);
tblOrders = new DataTable("tblOrders");
myDA2.Fill(tblOrders);
SqlDataAdapter myDA3 = new SqlDataAdapter("select a.order_id, a.line_item_number, a.product_id, b.name, a.quantity, a.extended_price " +
                                          "from line_items a, products b " +
                                          "where a.product_id = b.product_id " +
                                          "order by a.order_id, a.line_item_number", conn);
tblLineItems = new DataTable("tblLineItems");
myDA3.Fill(tblLineItems);

tblDataSet = new DataSet();

// load the Tables collection
tblDataSet.Tables.Add(tblCustomers);
tblDataSet.Tables.Add(tblOrders);
tblDataSet.Tables.Add(tblLineItems);

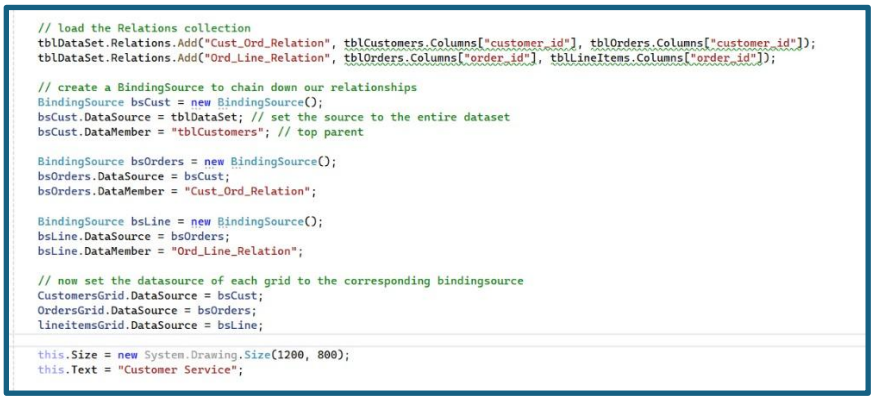
// load the Relations collection
tblDataSet.Relations.Add("Cust_Ord_Relation", tblCustomers.Columns["customer_id"], tblOrders.Columns["customer_id"]);
tblDataSet.Relations.Add("Ord_Line_Relation", tblOrders.Columns["order_id"], tblLineItems.Columns["order_id"]);

this.Size = new System.Drawing.Size(1200, 800);
this.Text = "Customer Service";

DataGridView CustomersGrid = new DataGridView();
CustomersGrid.Name = "customersGridView";
CustomersGrid.BorderStyle = BorderStyle.FixedSingle;
```

We next introduce the BindingSource class. This one is nice because you can specify both a DataSource and a DataMember. Think of a ComboBox where the DataSource is the displayed value and the DataMember is the value managed in the code. Same thing going on here. The DataSource is the more global source of data whereas the DataMember specifies which component to use within the source code. (Figure 21.6)

Figure 21.6: BindingSource



```
// load the Relations collection
tblDataSet.Relations.Add("Cust_Ord_Relation", tblCustomers.Columns["customer_id"], tblOrders.Columns["customer_id"]);
tblDataSet.Relations.Add("Ord_Line_Relation", tblOrders.Columns["order_id"], tblLineItems.Columns["order_id"]);

// create a BindingSource to chain down our relationships
BindingSource bsCust = new BindingSource();
bsCust.DataSource = tblDataSet; // set the source to the entire dataset
bsCust.DataMember = "tblCustomers"; // top parent

BindingSource bsOrders = new BindingSource();
bsOrders.DataSource = bsCust;
bsOrders.DataMember = "Cust_Ord_Relation";

BindingSource bsLine = new BindingSource();
bsLine.DataSource = bsOrders;
bsLine.DataMember = "Ord_Line_Relation";

// now set the datasource of each grid to the corresponding bindingsource
CustomersGrid.DataSource = bsCust;
OrdersGrid.DataSource = bsOrders;
lineitemsGrid.DataSource = bsLine;

this.Size = new System.Drawing.Size(1200, 800);
this.Text = "Customer Service";
```

Test it out. Your screen should now contain three synchronized data grids. Change the selected customer and the orders and lineitems for that customer are displayed. Change the selected order and the lineitems for that order are displayed. Come on, you know that is very cool.

We next have to deal with that product_id column in the lineitems grid. Nobody is going to memorize an entire catalog of products so we are going to need to add a ComboBox so the user may simply select a product name from the list. Our new code is shown here. (Figure 21.7) Please note that I have added the DataTable tblProducts to our global variable list up above, but you can't see that in this image. I also had to add the System.Collections namespace to my code up at the top.

Figure 21.7: Add ComboBox



```
myDA2.Fill(tblOrders);
SqlDataAdapter myDA3 = new SqlDataAdapter("select a.order_id, a.line_item_number, a.product_id, b.name, a.quantity, a.extended_price " +
                                          "from line_items a, products b " +
                                          "where a.product_id = b.product_id " +
                                          "order by a.order_id, a.line_item_number", conn);
tblLineItems = new DataTable("tblLineItems");
myDA3.Fill(tblLineItems);

SqlDataAdapter myDA4 = new SqlDataAdapter("select product_id, name from products order by product_id", conn);
tblProducts = new DataTable("tblProducts");
myDA4.Fill(tblProducts);

// add a column to the lineitems grid
lineitemsGrid.Columns.Add("Product Name", "Product");

// DropDown wants an Array as its DataSource
ArrayList StringList = new ArrayList();

// populate the ArrayList from tblProducts
foreach(DataRow row in tblProducts.Rows) StringList.Add(row["name"].ToString);

// now add a DropDown to that new lineitem grid column
for (int i = 0; i < lineitemsGrid.Rows.Count-1; i++)
{
    var myCell = new DataGridViewComboBoxCell();
    myCell.DataSource = StringList;
    lineitemsGrid.Rows[i].Cells[6] = myCell;
}

tblDataSet = new DataSet();
```

class System.String
Represents text as a sequence of UTF-16 code units.

One more thing needs to get done to our ComboBox code before we are ready for a test. We need to set the selected value of that guy equal to our original product name column. The associated code is shown here. (Figure 21.8)

Figure 21.8: Set Selected Value

```
// add a column to the lineitems grid
lineitemsGrid.Columns.Add("Product Name", "Product");

// DropDown wants an Array as its DataSource
ArrayList StringList = new ArrayList();

// populate the ArrayList from tblProducts
foreach(DataRow row in tblProducts.Rows) StringList.Add(row["name"].ToString);

// now add a DropDown to that new lineitem grid column
for (int i = 0; i < lineitemsGrid.Rows.Count-1; i++)
{
    var myCell = new DataGridViewComboBoxCell();
    myCell.DataSource = StringList;
    lineitemsGrid.Rows[i].Cells[6] = myCell;
}

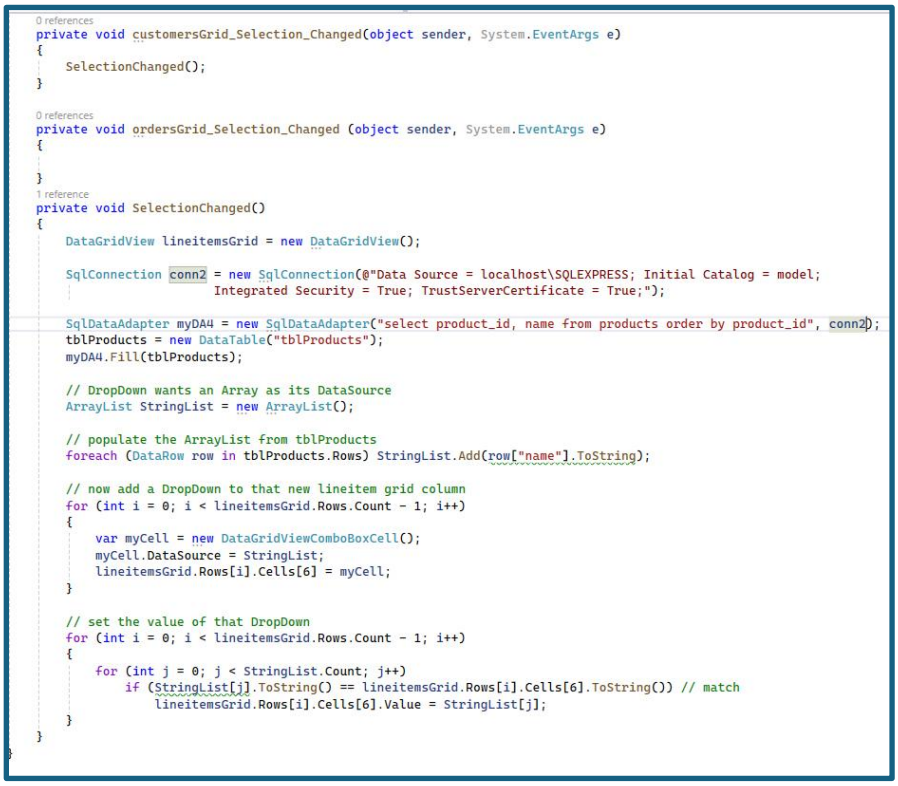
// set the value of that DropDown
for (int i = 0; i < lineitemsGrid.Rows.Count-1;i++)
{
    for (int j = 0; j < StringList.Count; j++)
        if (StringList[j].ToString() == lineitemsGrid.Rows[i].Cells[6].ToString()) // match
            lineitemsGrid.Rows[i].Cells[6].Value = StringList[j];
}

tblDataSet = new DataSet();
```

All right I just tested my code and my ComboBoxes are not showing up when I change the selected row in either the customers or orders grid. Why not? Because our original ComboBoxes were placed into the lineitems grid as a function of its Row collection, not the underlying DataSet. Time to get that fixed.

The event we need to be handling here is the SelectionChanged event of the customers and orders grid. So, what needs to happen when any of these values change? We need to rebuild our ComboBoxes in the lineitems grid. Well rather than write that code multiple times I simply pull that original code out into a new and separate routine aptly named SelectionChanged and point the original load code and both event handlers at it. The resulting code is shown here. (Figure 21.9) Please also note how I had to move the tblProducts Fill routine down into that SelectionChanged routine because that DataTable was not available to the grid change events.

Figure 21.9: SelectionChanged



```
0 references
private void customersGrid_Selection_Changed(object sender, System.EventArgs e)
{
    SelectionChanged();
}

0 references
private void ordersGrid_Selection_Changed (object sender, System.EventArgs e)
{

}
1 reference
private void SelectionChanged()
{
    DataGridView lineitemsGrid = new DataGridView();

    SqlConnection conn2 = new SqlConnection(@"Data Source = localhost\SQLEXPRESS; Initial Catalog = model;
                          Integrated Security = True; TrustServerCertificate = True;");

    SqlDataAdapter myDA4 = new SqlDataAdapter("select product_id, name from products order by product_id", conn2);
    tblProducts = new DataTable("tblProducts");
    myDA4.Fill(tblProducts);

    // DropDown wants an Array as its DataSource
    ArrayList StringList = new ArrayList();

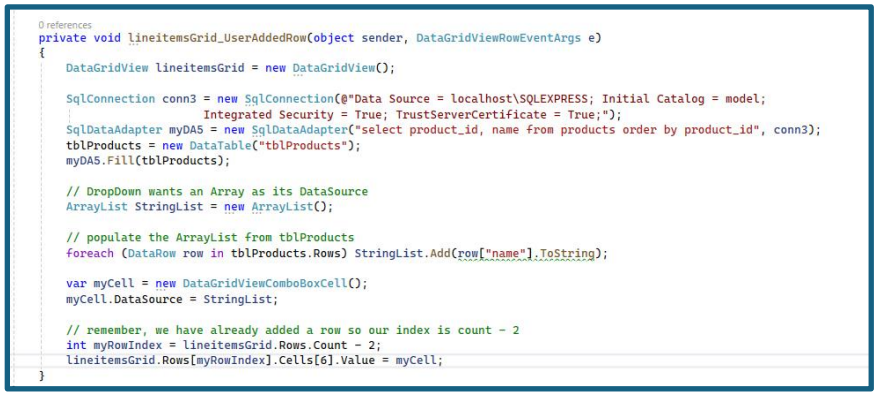
    // populate the ArrayList from tblProducts
    foreach (DataRow row in tblProducts.Rows) StringList.Add(row["name"].ToString);

    // now add a DropDown to that new lineitem grid column
    for (int i = 0; i < lineitemsGrid.Rows.Count - 1; i++)
    {
        var myCell = new DataGridViewComboBoxCell();
        myCell.DataSource = StringList;
        lineitemsGrid.Rows[i].Cells[6] = myCell;
    }

    // set the value of that DropDown
    for (int i = 0; i < lineitemsGrid.Rows.Count - 1; i++)
    {
        for (int j = 0; j < StringList.Count; j++)
            if (StringList[j].ToString() == lineitemsGrid.Rows[i].Cells[6].ToString()) // match
                lineitemsGrid.Rows[i].Cells[6].Value = StringList[j];
    }
}
```

We next need to consider the case where a user adds a new row to our lineitems grid. New rows in that guy are going to require a products ComboBox. My version of the code is shown here. (Figure 21.10) Only thing to remember here is that our current index is going to be Rows.Count-2 because we have just added a row.

Figure 21.10: UserAddedRow



```
0 references
private void lineitemsGrid_UserAddedRow(object sender, DataGridViewRowEventArgs e)
{
    DataGridView lineitemsGrid = new DataGridView();

    SqlConnection conn3 = new SqlConnection(@"Data Source = localhost\SQLEXPRESS; Initial Catalog = model;
                          Integrated Security = True; TrustServerCertificate = True;");
    SqlDataAdapter myDA5 = new SqlDataAdapter("select product_id, name from products order by product_id", conn3);
    tblProducts = new DataTable("tblProducts");
    myDA5.Fill(tblProducts);

    // DropDown wants an Array as its DataSource
    ArrayList StringList = new ArrayList();

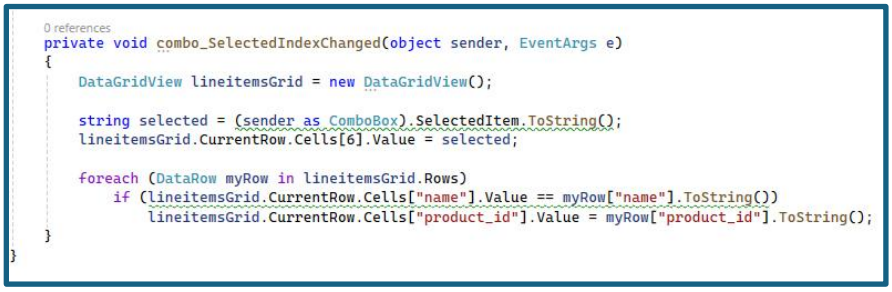
    // populate the ArrayList from tblProducts
    foreach (DataRow row in tblProducts.Rows) StringList.Add(row["name"].ToString);

    var myCell = new DataGridViewComboBoxCell();
    myCell.DataSource = StringList;

    // remember, we have already added a row so our index is count - 2
    int myRowIndex = lineitemsGrid.Rows.Count - 2;
    lineitemsGrid.Rows[myRowIndex].Cells[6].Value = myCell;
}
```

We next need to consider the case where the selected row in the lineitems grid changes. This is going to require a resynchronization of the product_id with the displayed product name. My version of this code is shown here. (Figure 21.11)

Figure 21.11: SelectedIndexChanged



```
0 references
private void combo_SelectedIndexChanged(object sender, EventArgs e)
{
    DataGridView lineitemsGrid = new DataGridView();

    string selected = (sender as ComboBox).SelectedItem.ToString();
    lineitemsGrid.CurrentRow.Cells[6].Value = selected;

    foreach (DataRow myRow in lineitemsGrid.Rows)
        if (lineitemsGrid.CurrentRow.Cells["name"].Value == myRow["name"].ToString())
            lineitemsGrid.CurrentRow.Cells["product_id"].Value = myRow["product_id"].ToString();
}
}
```

That concludes our code which manages the synchronization of the grids. Now it's time to start working on that submit button.

Now if you read the Microsoft docs they will tell you that you should be updating the database every time someone inserts, updates or deletes a row from your grids. As someone who designed and maintained databases for many years, I am telling you that Microsoft is absolutely WRONG! Making a database update is a very expensive action. Much more expensive than a database read. Solution - run a single set of database updates off of the submit button versus doing independent ones each time a change is made and you will get much better application scalability.

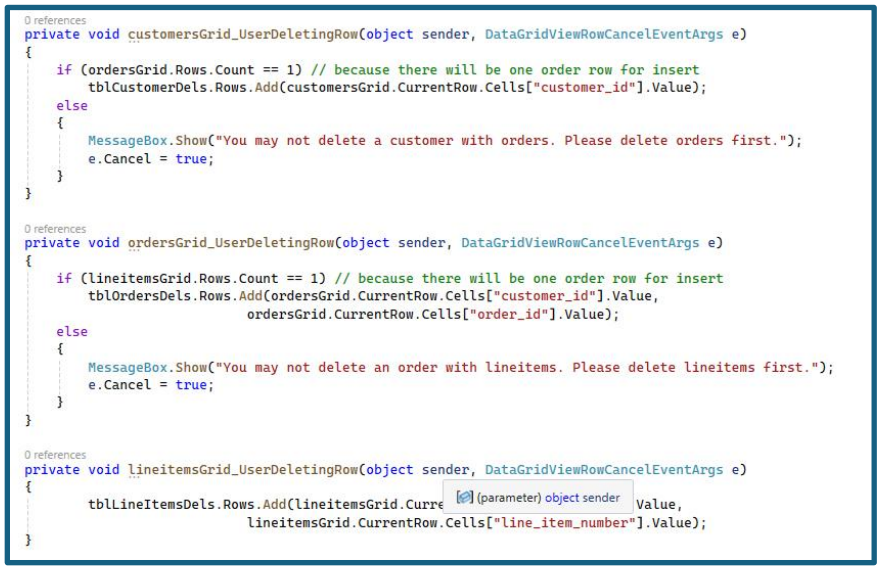
I can now get off my soapbox and get back to our code. So, if I am going to make just a single set of database updates rather than multiple independent ones how am I going to know which grid rows were deleted? The answer is given in Figure 21.12 with the introduction of three new DataTables, one for each grid. These guys will be used to store the primary key segments of deleted rows in each of their corresponding data grids. That code is shown in Figure 21.13. Please note that the UserDeletingRow event fires before the row is deleted thus allowing for this implementation.

Figure 21.12: Delete Tables



```
3 references
public partial class Form1 : Form
{
    DataTable tblCustomers, tblOrders, tblLineItems, tblProducts;
    DataSet tblDataSet;
    DataTable tblCustomerDels, tblOrderDels, tblLineItemDels;
    1 reference
    public Form1()
    {
        InitializeComponent();
    }
```

Figure 21.13: UserDeletingRow



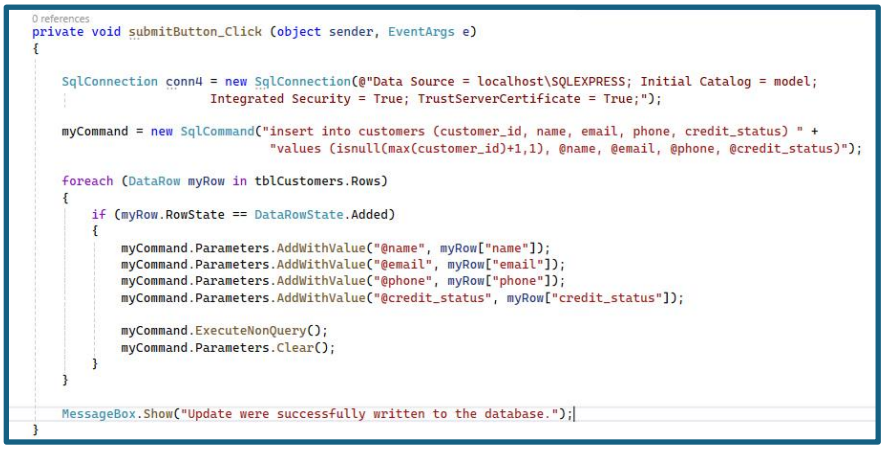
```
0 references
private void customersGrid_UserDeletingRow(object sender, DataGridViewRowCancelEventArgs e)
{
    if (ordersGrid.Rows.Count == 1) // because there will be one order row for insert
        tblCustomerDels.Rows.Add(customersGrid.CurrentRow.Cells["customer_id"].Value);
    else
    {
        MessageBox.Show("You may not delete a customer with orders. Please delete orders first.");
        e.Cancel = true;
    }
}

0 references
private void ordersGrid_UserDeletingRow(object sender, DataGridViewRowCancelEventArgs e)
{
    if (lineitemsGrid.Rows.Count == 1) // because there will be one order row for insert
        tblOrdersDels.Rows.Add(ordersGrid.CurrentRow.Cells["customer_id"].Value,
                               ordersGrid.CurrentRow.Cells["order_id"].Value);
    else
    {
        MessageBox.Show("You may not delete an order with lineitems. Please delete lineitems first.");
        e.Cancel = true;
    }
}

0 references
private void lineitemsGrid_UserDeletingRow(object sender, DataGridViewRowCancelEventArgs e)
{
        tblLineItemsDels.Rows.Add(lineitemsGrid.Curre (parameter) object sender Value,
                               lineitemsGrid.CurrentRow.Cells["line_item_number"].Value);
}
```

It is finally time to handle that submit button. The magic we are going to utilize is the fact that each of our DataTable DataRow objects has a RowState property which assumes a value based on updates to the underlying DataTable. For example, to determine whether a DataRow was added to a DataTable check its RowState property against the DataRowState.Added enum. Please remember that when you are doing inserts that you must start at the root parent as shown here. Please also note that to save time in this already long lesson we are only going to review the customer table inserts. The other two tables and all the updates of course follow the exact same logic. (Figure 21.14)

Figure 21.14: submitButton_Click



```
0 references
private void submitButton_Click (object sender, EventArgs e)
{

    SqlConnection conn4 = new SqlConnection(@"Data Source = localhost\SQLEXPRESS; Initial Catalog = model;
                        Integrated Security = True; TrustServerCertificate = True;");

    myCommand = new SqlCommand("insert into customers (customer_id, name, email, phone, credit_status) " +
                               "values (isnull(max(customer_id)+1,1), @name, @email, @phone, @credit_status)");

    foreach (DataRow myRow in tblCustomers.Rows)
    {
        if (myRow.RowState == DataRowState.Added)
        {
            myCommand.Parameters.AddWithValue("@name", myRow["name"]);
            myCommand.Parameters.AddWithValue("@email", myRow["email"]);
            myCommand.Parameters.AddWithValue("@phone", myRow["phone"]);
            myCommand.Parameters.AddWithValue("@credit_status", myRow["credit_status"]);

            myCommand.ExecuteNonQuery();
            myCommand.Parameters.Clear();
        }
    }

    MessageBox.Show("Update were successfully written to the database.");
}
```

This concludes our review of manual database reads and updates through C#. In our next lesson we are going to take a look at database stored procedures. An alternative way to make database updates which is often faster and more efficient.